PUNJAB UNIVERSITY COLLEGE OF INFORMATION TECHNOLOGY

Project Topic:

ADMISSION MANAGEMENT SYSTEM

Course Name:

DATABASE SYSTEM

Submitted to:

DR.ASIF SOHAIL

Submitted By:

ROLL NUMBER	NAME
BITF24M008	Shanza Shafiq
BITF24M016	Aswah Rani
BITF24M029	Tehreem Fatima

1. INTRODUCTION

The **Admission Management System** is a computerized solution designed to automate and streamline the entire student admission process of a university. The system replaces the traditional manual admission method with a fast, transparent, and user-friendly online portal. It allows students to apply for different programs, upload required documents, provide academic and test information, and track the status of their application in real time.

The software supports all major admission categories including **Open Merit**, **Self-Support**, and **Quota-Based Admissions** such as Sports Quota, Disabled Quota, Minority Quota, Hafiz-e-Quran, and others. The system also handles **entry test marks**, **matric & intermediate marks**, and automatically calculates the **final merit score** by applying the university's weightage criteria.

It also includes additional utilities such as **Hostel Management**, allowing students to apply for hostel accommodation with a simple online request. Fee payments such as application fee, admission fee, self-support charges, and hostel fee are recorded and verified through the integrated account office module.

The Admission Management System ensures accuracy, reduces workload on staff, minimizes human error, and brings transparency to the selection process. It is a complete digital solution suitable for modern educational institutions.

WORKING OF THE SYSTEM

1. User Registration & Login

- A student creates an online account.
- Login credentials are generated.
- Profile information is completed (personal, contact, email etc.)

2. Application Form

Student fills and submits the online application by selecting:

- Campus
- Session
- Department
- Program

- Admission Category
    - Open Merit
    - Self Support
    - Quota
- Quota Type (only if quota selected)
- Hostel Request Option

The system stores all data in the **Application** table.

3. Academic Information Submission

Student enters:

- Matric Total + Obtained Marks
- Intermediate Total + Obtained Marks
- Percentage auto-calculated

Records saved to **AcademicMarks** table.

4. Entry Test Marks

The student takes the university's admission test.
Admin adds:

- Test Name
- Test Date
- Total Marks
- Obtained Marks

Stored in **TestScores** table.

5. Hafiz-e-Quran Marks (Bonus)

If the applicant is Hafiz-e-Quran:

- Upload certificate
- Admin verifies
- System adds **+20 bonus marks** during merit calculation.

Stored in **HafizCertificate** table.

6. Document Upload

Student uploads:

- CNIC/B-Form
- Picture
- Certificates
- Quota proof documents
- Medical certificate
- Domicile

Admin verifies documents through the **Documents** table.

7. Fee Management

Account office handles:

- Application Fee
- Admission Fee
- Self-Support Fee
- Hostel fee

Students upload bank slip or use online payment.
Admin verifies and marks payment status as **confirmed**.

Stored in **Payments** table.

8. Merit Calculation

System automatically calculates the final merit using:

- Entry Test Weightage
- Intermediate Weightage
- Matric Weightage
- Hafiz Bonus Marks

After scoring:

- Students are ranked
- Merit List is generated
- Open Merit, Quota, and Self-Support seats are allocated separately

Stored in **Merit** table.

9. Admission Offer

Based on merit list:

- System marks status as **Offered** for selected students
- Others remain **Waiting** or **Not Selected**

Students can accept the offer.

10. Hostel Management (Single Entity)

If student requested hostel:

- Admin checks hostel availability
- Hostel fee added
- If needed, student may request **cancellation**

All stored in the **Hostel** table (single entity).

11. Final Admission Approval

After:

- Document verification
- Fee confirmation
- Merit allocation
- Hostel approval (if applicable)
- Issued challen

Admin changes application status to:

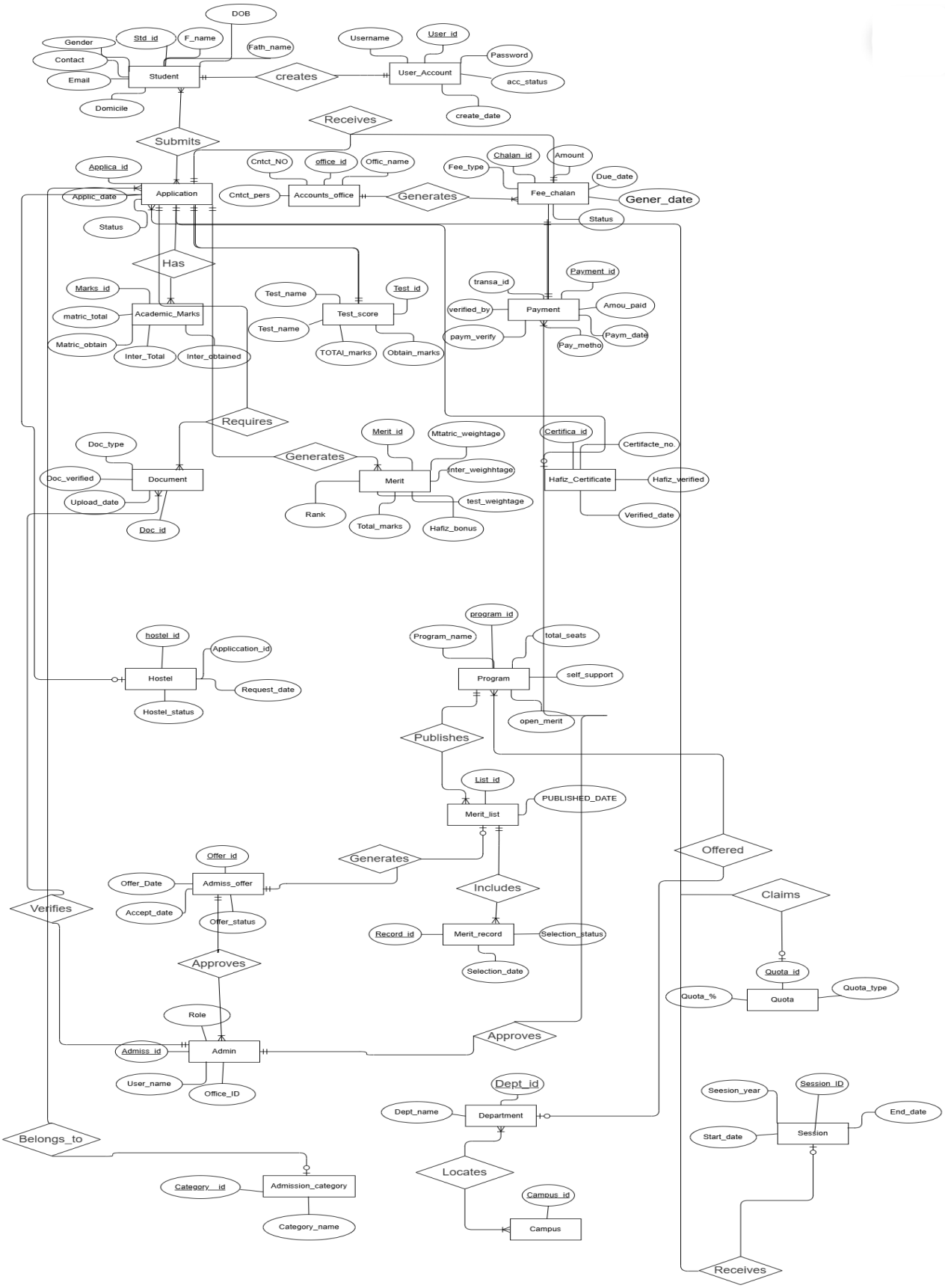
12. **Admitted**

The student is officially offered admission into the university.

2. ERD

Entity 1	Relationship	Entity 2	Connectivity	Description
STUDENT	CREATES	USER_ACCOUNT	1 : 1	Student login account
STUDENT	SUBMITS	APPLICATION	1 : M	Multiple applications
ACCOUNTS_OFFICE	GENERATES	FEE_CHALLAN	1 : M	NEW: Fee challan generation
FEE_CHALLAN	PAID_VIA	PAYMENT	1 : M	NEW: Challan payment link
APPLICATION	HAS	ACADEMIC_MARKS	1 : 1	Academic record
APPLICATION	HAS	TEST_SCORE	1 : 1	Test score
APPLICATION	HAS	HAFIZ_CERTIFICATE	1 : 0..1	Optional certificate
APPLICATION	REQUIRES	DOCUMENTS	M : N	Multiple documents
APPLICATION	RECEIVES	FEE_CHALLAN	1 : M	NEW: Application gets challans
APPLICATION	GENERATES	MERIT	1 : 1	Merit score
APPLICATION	REQUESTS	HOSTEL	M : 0..1	Optional hostel
PROGRAM	PUBLISHES	MERIT_LIST	1 : M	Program merit list
MERIT_LIST	INCLUDES	MERIT_RECORD	1 : M	Merit records
MERIT_LIST	GENERATES	ADMISSION_OFFER	1 : M	Admission offers
ADMIN	VERIFIES	DOCUMENTS	1 : M	Document verification
ADMIN	VERIFIES	PAYMENT	1 : M	Payment verification
ADMIN	APPROVES	ADMISSION_OFFER	1 : M	Final approval

APPLICATION	BELONGS_TO	ADMISSION_CATEGORY	M : 1	Category
APPLICATION	CLAIMS	QUOTA	M : 0..1	Optional quota
SESSION	RECEIVES	APPLICATION	1 : M	Session applications
PROGRAM	OFFERED_BY	DEPARTMENT	M : 1	Department programs
DEPARTMENT	LOCATED_IN	CAMPUS	M : 1	Campus


DOB
Gender
Std_id
F_name
Fath_name
Contact
Email
Student
creates
Username
User_id
User_Account
Password
acc_status
create_date
Domicile
Receives
Submits
Applica_id
Cntct_NO
office_id
Offic_name
Fee_type
Chalan_id
Amount
Due_date
Applic_date
Application
Cntct_pers
Accounts_office
Generates
Fee_chalan
Gener_date
Status
Status
Has
Payment_id
transa_id
Marks_id
Test_name
Test_id
verified_by
Payment
Amou_paid
matric_total
Academic_Marks
Test_score
Test_name
paym_verify
Paym_date
Matric_obtain
Inter_Total
Inter_obtained
TOTAl_marks
Obtain_marks
Pay_metho
Requires
Doc_type
Merit_id
Mtatric_weightage
Certifica_id
Certifacte_no
Generates
Doc_verified
Document
Merit
Inter_weighhtage
Hafiz_Certificate
Hafiz_verified
Upload_date
Rank
test_weightage
Verified_date
Total_marks
Hafiz_bonus
Doc_id
program_id
hostel_id
Applicaation_id
Program_name
total_seats
Hostel
Program
self_support
Request_date
Hostel_status
open_merit
Publishes
List_id
PUBLISHED_DATE
Merit_list
Offer_id
Generates
Offered
Offer_Date
Admiss_offer
Includes
Claims
Verifies
Accept_date
Offer_status
Record_id
Merit_record
Selection_status
Selection_date
Approves
Quota_id
Quota_%
Quota
Quota_type
Role
Approves
Admiss_id
Admin
User_name
Office_ID
Dept_id
Seesion_year
Session_ID
Dept_name
Department
End_date
Belongs_to
Start_date
Session
Locates
Campus_id
Category__id
Admission_category
Campus
Category_name
Receives

3. CONSTRUCTION OF THE RELATIONAL SCHEMA

BOTOTM UP APPROACH

STEP 1. IDENTIFICATION OF ENTITIES AND ATTRIBUTES:

One entity, Admission Management System, with list of attributes:

(STUDENT_ID, CNIC, FULL_NAME, FATHER_NAME, DOB, GENDER, CONTACT, EMAIL, DOMICILE,USER_ID, STUDENT_ID, USERNAME, PASSWORD_HASH, ACCOUNT_STATUS, CREATED_DATE,OFFICE_ID, OFFICE_NAME, CAMPUS_ID, CONTACT_PERSON, CONTACT_NUMBER,CAMPUS_ID, CAMPUS_NAME, LOCATION,DEPT_ID, DEPT_NAME, CAMPUS_ID,PROGRAM_ID, PROGRAM_NAME, DEPT_ID, TOTAL_SEATS,OPEN MERIT SEATS, SELF_SUPPORT_SEATS,SESSION_ID, SESSION_YEAR, START_DATE, END_DATE,CATEGORY_ID, CATEGORY_NAME,QUOTA_ID, QUOTA_TYPE, QUOTA_PERCENTAGE,APPLICATION_ID, STUDENT_ID, SESSION_ID, PROGRAM_ID, CATEGORY_ID, QUOTA_ID,APPLICATION_DATE, STATUS,MARKS_ID, APPLICATION_ID, MATRIC_TOTAL, MATRIC_OBTAINED, INTER_TOTAL, INTER_OBTAINED,TEST_ID, APPLICATION_ID, TEST_NAME, TEST_DATE, TOTAL_MARKS, OBTAINED_MARKS,CERTIFICATE_ID, AP PLICATION_ID, CERTIFICATE_NUMBER, HAFIZ_VERIFIED, VERIFIED_DATE,DOC_ID, APPLICATION_ID, DOC_TYPE, FILE_PATH, UPLOADED_DATE, DOC_VERIFIED, VERIFIED_BY,CHALLAN_ID, APPLICATION_ID, OFFICE_ID, FEE_TYPE, AMOUNT, DUE_DATE, GENERATED_DATE, STATUS,PAYMENT_ID, CHALLAN_ID, PAYMENT_DATE, AMOUNT_PAID, PAYMENT_METHOD, TRANSACTION_ID,PAYMENT_VERIFIED, VERIFIED_BY,MERIT_ID, APPLICATION_ID, MATRIC_WEIGHTAGE, INTER_WEIGHTAGE, TEST_WEIGHTAGE,HAFIZ_BONUS, TOTAL_SCORE, RANK,HOSTEL_ID, APPLICATION_ID, REQUEST_DATE, HOSTEL_STATUS,LIST_ID, PROGRAM_ID, SESSION_ID, CATEGORY_ID, PUBLISHED_DATE,RECORD_ID, LIST_ID, MERIT_ID, SELECTION_STATUS, SELECTION_DATE,OFFER_ID, APPLICATION_ID, OFFER_DATE, ACCEPTANCE_DEADLINE, OFFER_STATUS,ADMIN_ID, USERNAME, ROLE, OFFICE_ID)

STEP 2. IDENTIFICATION OF FUNCTIONAL DEPENDENCIES:

Following different determinants (forming group of related information) can be separated out from this list of attributes, which are giving rise to different functional dependencies:

Student personal information, User account detail, Campus and department structure, Academic programs and seats, Admission sessions and categories, Application submission and processing, Academic marks and test scores, Document verification and certificates, Fee challans and payments, Merit calculation and ranking, Hostel accommodation requests, Merit lists and selection records, Admission offers and decisions, Administrative controls and offices.

STEP 3. GROUPING OF ATTRIBUTES INTO RELATIONS:

Based upon the above attributes and entities, we can break down the list of attributes into following relations:

Student & Account Management:

STUDENT: STUDENT_ID, CNIC, FULL_NAME, FATHER_NAME, DOB, GENDER, CONTACT, EMAIL, DOMICILE

USER_ACCOUNT: USER_ID, STUDENT_ID, USERNAME, PASSWORD_HASH, ACCOUNT_STATUS, CREATED_DATE

ADMIN: ADMIN_ID, USERNAME, ROLE, OFFICE_ID

University Structure:

CAMPUS: CAMPUS_ID, CAMPUS_NAME, LOCATION

DEPARTMENT: DEPT_ID, DEPT_NAME, CAMPUS_ID

PROGRAM: PROGRAM_ID, PROGRAM_NAME, DEPT_ID, TOTAL_SEATS, SELF_SUPPORT_SEATS,OPEN_MERIT_ SEATS

ACCOUNTS_OFFICE: OFFICE_ID, OFFICE_NAME, CAMPUS_ID, CONTACT_PERSON, CONTACT_NUMBER

Admission Setup:

SESSION: SESSION_ID, SESSION_YEAR, START_DATE, END_DATE

ADMISSION_CATEGORY: CATEGORY_ID, CATEGORY_NAME

QUOTA: QUOTA_ID, QUOTA_TYPE, QUOTA_PERCENTAGE

Application Process:

APPLICATION: APPLICATION_ID, STUDENT_ID, SESSION_ID, PROGRAM_ID, CATEGORY_ID, QUOTA_ID, APPLICATION_DATE, STATUS

ACADEMIC_MARKS: MARKS_ID, APPLICATION_ID, MATRIC_TOTAL, MATRIC_OBTAINED, INTER_TOTAL, INTER_OBTAINED

TEST_SCORE: TEST_ID, APPLICATION_ID, TEST_NAME, TEST_DATE, TOTAL_MARKS, OBTAINED_MARKS

HAFIZ_CERTIFICATE: CERTIFICATE_ID, APPLICATION_ID, CERTIFICATE_NUMBER, HAFIZ_VERIFIED, VERIFIED_DATE

DOCUMENTS: DOC_ID, APPLICATION_ID, DOC_TYPE, FILE_PATH, UPLOADED_DATE, DOC_VERIFIED, VERIFIED_BY

Financial Management:

FEE_CHALLAN: CHALLAN_ID, APPLICATION_ID, OFFICE_ID, FEE_TYPE, AMOUNT, DUE_DATE, GENERATED_DATE, STATUS

PAYMENT: PAYMENT_ID, CHALLAN_ID, PAYMENT_DATE, AMOUNT_PAID, PAYMENT_METHOD, TRANSACTION_ID, PAYMENT_VERIFIED, VERIFIED_BY

Merit & Selection:

MERIT: MERIT_ID, APPLICATION_ID, MATRIC_WEIGHTAGE, INTER_WEIGHTAGE, TEST_WEIGHTAGE, HAFIZ_BONUS, TOTAL_SCORE, RANK

MERIT_LIST: LIST_ID, PROGRAM_ID, SESSION_ID, CATEGORY_ID, PUBLISHED_DATE

MERIT_RECORD: RECORD_ID, LIST_ID, MERIT_ID, SELECTION_STATUS, SELECTION_DATE

ADMISSION_OFFER: OFFER_ID, APPLICATION_ID, OFFER_DATE, ACCEPTANCE_DEADLINE, OFFER_STATUS

Additional Services:

HOSTEL: HOSTEL_ID, APPLICATION_ID, REQUEST_DATE, HOSTEL_STATUS

STEP 4: NORMALIZATION PROCESS

STUDENT Table

(STUDENT_ID, CNIC, FULL_NAME, FATHER_NAME, DOB, GENDER, CONTACT, EMAIL, DOMICILE)

- **1NF:** All attributes contain atomic values and there are no repeating groups.
- **2NF:** Since the table has a single primary key, there are no partial dependencies.
- **3NF:** There are no transitive dependencies among non-key attributes.

- **BCNF:** STUDENT_ID and CNIC both act as candidate keys.

USER_ACCOUNT Table

(USER_ID, STUDENT_ID, USERNAME, PASSWORD_HASH, ACCOUNT_STATUS, CREATED_DATE)

- **1NF:** All attributes are atomic and no repeating groups exist.
- **2NF:** The table has a single primary key, therefore no partial dependency exists.
- **3NF:** There are no transitive dependencies in the table.
- **BCNF:** Multiple candidate keys exist in this table.

CAMPUS Table

(CAMPUS_ID, CAMPUS_NAME, LOCATION)

- **1NF:** All attributes contain atomic values.
- **2NF:** Since there is a single primary key, no partial dependency exists.
- **3NF:** No transitive dependency exists among non-key attributes.
- **BCNF:** The table has a single candidate key.

DEPARTMENT Table

(DEPT_ID, DEPT_NAME, CAMPUS_ID)

- **1NF:** All attributes are atomic and non-repeating.
- **2NF:** The table has a single primary key, so partial dependency does not exist.
- **3NF:** There are no transitive dependencies.
- **BCNF:** A single candidate key exists.

PROGRAM Table

(PROGRAM_ID, PROGRAM_NAME, DEPT_ID, TOTAL_SEATS, SELF_SUPPORT_SEATS,OPEN_MERIT_SEATS)

- **1NF:** All attributes contain atomic values.
- **2NF:** There is no partial dependency due to a single primary key.
- **3NF:** No transitive dependency exists.
- **BCNF:** Two candidate keys exist in this table.

SESSION Table

(SESSION_ID, SESSION_YEAR, START_DATE, END_DATE)

- **1NF:** All attributes are atomic.
- **2NF:** The table has a single primary key, hence no partial dependency.
- **3NF:** There are no transitive dependencies.
- **BCNF:** A single candidate key exists.

ADMISSION_CATEGORY Table

(CATEGORY_ID, CATEGORY_NAME)

- **1NF:** All attributes contain atomic values.
- **2NF:** No partial dependency exists due to a single primary key.
- **3NF:** No transitive dependency exists.
- **BCNF:** The table has a single candidate key.

QUOTA Table

(QUOTA_ID, QUOTA_TYPE, QUOTA_PERCENTAGE)

- **1NF:** All attributes are atomic.
- **2NF:** The table has a single primary key, so no partial dependency exists.
- **3NF:** There are no transitive dependencies.
- **BCNF:** Two candidate keys exist.

APPLICATION Table

(APPLICATION_ID, STUDENT_ID, SESSION_ID, PROGRAM_ID, CATEGORY_ID, QUOTA_ID, APPLICATION_DATE, STATUS)

- **1NF:** All attributes contain atomic values.
- **2NF:** Since the primary key is single, partial dependency does not exist.
- **3NF:** There are no transitive dependencies.
- **BCNF:** Multiple candidate keys exist in this table.

ACADEMIC_MARKS Table

(MARKS_ID, APPLICATION_ID, MATRIC_TOTAL, MATRIC_OBTAINED, INTER_TOTAL, INTER_OBTAINED)

- **1NF:** All attributes are atomic.
- **2NF:** No partial dependency exists due to a single primary key.
- **3NF:** There are no transitive dependencies.
- **BCNF:** Two candidate keys exist.

TEST_SCORE Table

(TEST_ID, APPLICATION_ID, TEST_NAME, TEST_DATE, TOTAL_MARKS, OBTAINED_MARKS)

- **1NF:** All attributes contain atomic values.
- **2NF:** The table has a single primary key, so no partial dependency exists.
- **3NF:** No transitive dependency exists.
- **BCNF:** A single candidate key exists.

HAFIZ_CERTIFICATE Table

(CERTIFICATE_ID, APPLICATION_ID, CERTIFICATE_NUMBER, HAFIZ_VERIFIED, VERIFIED_DATE)

- **1NF:** All attributes are atomic.
- **2NF:** No partial dependency exists due to a single primary key.
- **3NF:** There are no transitive dependencies.
- **BCNF:** Two candidate keys exist.

DOCUMENTS Table

(DOC_ID, APPLICATION_ID, DOC_TYPE, FILE_PATH, UPLOADED_DATE, DOC_VERIFIED, VERIFIED_BY)

- **1NF:** All attributes contain atomic values.
- **2NF:** The table has a single primary key, so no partial dependency exists.
- **3NF:** There are no transitive dependencies.
- **BCNF:** A single candidate key exists.

FEE_CHALLAN Table

(CHALLAN_ID, APPLICATION_ID, OFFICE_ID, FEE_TYPE, AMOUNT, DUE_DATE, GENERATED_DATE, STATUS)

- **1NF:** All attributes are atomic.
- **2NF:** No partial dependency exists due to a single primary key.
- **3NF:** There are no transitive dependencies.
- **BCNF:** A single candidate key exists.

PAYMENT Table

(PAYMENT_ID, CHALLAN_ID, PAYMENT_DATE, AMOUNT_PAID, PAYMENT_METHOD, TRANSACTION_ID, PAYMENT_VERIFIED, VERIFIED_BY)

- **1NF:** All attributes contain atomic values.
- **2NF:** The table has a single primary key, so no partial dependency exists.
- **3NF:** No transitive dependency exists.
- **BCNF:** Two candidate keys exist.

MERIT Table

(MERIT_ID, APPLICATION_ID, MATRIC_WEIGHTAGE, INTER_WEIGHTAGE, TEST_WEIGHTAGE, HAFIZ_BONUS, TOTAL_SCORE, RANK)

- **1NF:** All attributes are atomic.
- **2NF:** No partial dependency exists due to a single primary key.
- **3NF:** The table violates 3NF because RANK depends on TOTAL_SCORE, and TOTAL_SCORE depends on APPLICATION_ID.
- **BCNF:** The table does not satisfy BCNF.
- **SOLUTION**: To resolve this normalization violation, remove the RANK attribute from the MERIT table. RANK can be calculated dynamically during merit list generation using SQL window functions. After removing RANK, the table will satisfy both 3NF and BCNF.

HOSTEL Table

(HOSTEL_ID, APPLICATION_ID, REQUEST_DATE, HOSTEL_STATUS)

- **1NF:** All attributes contain atomic values.

- **2NF:** No partial dependency exists due to a single primary key.
- **3NF:** There are no transitive dependencies.
- **BCNF:** Two candidate keys exist.

MERIT_LIST Table

(LIST_ID, PROGRAM_ID, SESSION_ID, CATEGORY_ID, PUBLISHED_DATE)

- **1NF:** All attributes are atomic.
- **2NF:** The table has a single primary key, so no partial dependency exists.
- **3NF:** There are no transitive dependencies.
- **BCNF:** A composite candidate key exists.

MERIT_RECORD Table

(RECORD_ID, LIST_ID, MERIT_ID, SELECTION_STATUS, SELECTION_DATE)

- **1NF:** All attributes contain atomic values.
- **2NF:** No partial dependency exists due to a single primary key.
- **3NF:** There are no transitive dependencies.
- **BCNF:** A composite candidate key exists.

ADMISSION_OFFER Table

(OFFER_ID, APPLICATION_ID, OFFER_DATE, ACCEPTANCE_DEADLINE, OFFER_STATUS)

- **1NF:** All attributes are atomic.
- **2NF:** The table has a single primary key, so no partial dependency exists.
- **3NF:** No transitive dependency exists.
- **BCNF:** Two candidate keys exist.

ADMIN Table

(ADMIN_ID, USERNAME, ROLE, OFFICE_ID)

- **1NF:** All attributes contain atomic values.
- **2NF:** No partial dependency exists due to a single primary key.
- **3NF:** There are no transitive dependencies.

- **BCNF:** Two candidate keys exist.

ACCOUNTS_OFFICE Table

(OFFICE_ID, OFFICE_NAME, CAMPUS_ID, CONTACT_PERSON, CONTACT_NUMBER)

- **1NF:** All attributes are atomic.
- **2NF:** The table has a single primary key, so no partial dependency exists.
- **3NF:** There are no transitive dependencies.
- **BCNF:** A composite candidate key exists.

STEP 5. IDENTIFICATION OF CANDIDATE KEYS, PRIMARY KEYS, AND ALTERNATE KEYS:

Here are the Primary Keys (pK) presented in a table format:

Table	Primary Key
STUDENT	STUDENT_ID
USER_ACCOUNT	USER_ID
CAMPUS	CAMPUS_ID
DEPARTMENT	DEPT_ID
PROGRAM	PROGRAM_ID
SESSION	SESSION_ID
ADMISSION_CATEGORY	CATEGORY_ID
QUOTA	QUOTA_ID
APPLICATION	APPLICATION_ID
ACADEMIC_MARKS	MARKS_ID
TEST_SCORE	TEST_ID
HAFIZ_CERTIFICATE	CERTIFICATE_ID
DOCUMENTS	DOC_ID
FEE_CHALLAN	CHALLAN_ID
PAYMENT	PAYMENT_ID
MERIT	MERIT_ID
HOSTEL	HOSTEL_ID
MERIT_LIST	LIST_ID
MERIT_RECORD	RECORD_ID
ADMISSION_OFFER	OFFER_ID
ADMIN	ADMIN_ID
ACCOUNTS_OFFICE	OFFICE_ID

FOREIGN KEY

Here are the Foreign Keys (FK) presented in a table format:

Table	Foreign Key	References (Parent Table)
USER_ACCOUNT	STUDENT_ID	STUDENT(STUDENT_ID)
DEPARTMENT	CAMPUS_ID	CAMPUS(CAMPUS_ID)
PROGRAM	DEPT_ID	DEPARTMENT(DEPT_ID)
APPLICATION	STUDENT_ID	STUDENT(STUDENT_ID)
APPLICATION	SESSION_ID	SESSION(SESSION_ID)
APPLICATION	PROGRAM_ID	PROGRAM(PROGRAM_ID)
APPLICATION	CATEGORY_ID	ADMISSION_CATEGORY(CATEGORY_ID)
APPLICATION	QUOTA_ID	QUOTA(QUOTA_ID)
ACADEMIC_MARKS	APPLICATION_ID	APPLICATION(APPLICATION_ID)
TEST_SCORE	APPLICATION_ID	APPLICATION(APPLICATION_ID)
HAFIZ_CERTIFICATE	APPLICATION_ID	APPLICATION(APPLICATION_ID)
DOCUMENTS	APPLICATION_ID	APPLICATION(APPLICATION_ID)
FEE_CHALLAN	APPLICATION_ID	APPLICATION(APPLICATION_ID)
FEE_CHALLAN	OFFICE_ID	ACCOUNTS_OFFICE(OFFICE_ID)
PAYMENT	CHALLAN_ID	FEE_CHALLAN(CHALLAN_ID)
MERIT	APPLICATION_ID	APPLICATION(APPLICATION_ID)
HOSTEL	APPLICATION_ID	APPLICATION(APPLICATION_ID)
MERIT_LIST	PROGRAM_ID	PROGRAM(PROGRAM_ID)
MERIT_LIST	SESSION_ID	SESSION(SESSION_ID)
MERIT_LIST	CATEGORY_ID	ADMISSION_CATEGORY(CATEGORY_ID)
MERIT_RECORD	LIST_ID	MERIT_LIST(LIST_ID)
MERIT_RECORD	MERIT_ID	MERIT(MERIT_ID)
ADMISSION_OFFER	APPLICATION_ID	APPLICATION(APPLICATION_ID)
ADMIN	OFFICE_ID	ACCOUNTS_OFFICE(OFFICE_ID)
ACCOUNTS_OFFICE	CAMPUS_ID	CAMPUS(CAMPUS_ID)

ALTERNATE KEY

Here are the Alternate Keys (AK) presented in a table format:

Table	Alternate Key (AK)
STUDENT	CNIC
USER_ACCOUNT	(STUDENT_ID, USERNAME)
CAMPUS	None
DEPARTMENT	None
PROGRAM	(PROGRAM_NAME, DEPT_ID)

SESSION	None
ADMISSION_CATEGORY	None
QUOTA	QUOTA_TYPE
APPLICATION	(STUDENT_ID, SESSION_ID, PROGRAM_ID)
ACADEMIC_MARKS	APPLICATION_ID
TEST_SCORE	None
HAFIZ_CERTIFICATE	CERTIFICATE_NUMBER
DOCUMENTS	None
FEE_CHALLAN	None
PAYMENT	TRANSACTION_ID
MERIT	APPLICATION_ID
HOSTEL	APPLICATION_ID
MERIT_LIST	(PROGRAM_ID, SESSION_ID, CATEGORY_ID)
MERIT_RECORD	(LIST_ID, MERIT_ID)
ADMISSION_OFFER	APPLICATION_ID
ADMIN	USERNAME
ACCOUNTS_OFFICE	(OFFICE_NAME, CAMPUS_ID)

TOP DOWN APPROACH

1. REQUIREMENT ANALYSIS AND DATA COLLECTION

For the Admission Management System (AMS), comprehensive stakeholder analysis was conducted with students, parents, university administration, admission staff, accounts department, and hostel management.

Key requirements identified:

- Student personal, academic, and category details
- University structure: campuses, departments, programs, sessions
- Admission workflow from application to selection
- Financial management of fees and payments
- Supporting hostel and document services
- Administrative user roles and reporting

Current manual process inefficiencies in application handling, merit calculation, fee processing, and communication were analyzed. This ensures a transparent, efficient, and fair digital admission system.

2.CONCEPTUAL DESIGN (ER MODEL CREATION):

Given in part 2 of the Project-Report (see above)

3.MAPPING ER MODEL TO RELATIONAL SCHEMA

ER diagram is converted into tables as shown in part 4 and 5 of the Project-Report (see below).

4.Identification of Primary and Foreign Keys:

Keys are shown above .

5.Schema Refinement and Validation

Done through the process of normalization, similar to the process mentioned in part 3 (Bottom-Up Approach) of Project-Report.

---

4. DESCRIPTION OF TABLE

Table 1: STUDENT

Attribute	Data Type	Size	Constraints
STUDENT_ID	NUMBER	—	**Primary Key**
CNIC	VARCHAR2	15	**UNIQUE, NOT NULL**
FULL_NAME	VARCHAR2	50	**NOT NULL**
FATHER_NAME	VARCHAR2	50	—
DOB	DATE	—	**NOT NULL**
GENDER	VARCHAR2	10	CAN BE MALE AND FEMALE
CONTACT	VARCHAR2	15	11-DIGIT NUMERIC PAKISTANI FORMAT
EMAIL	VARCHAR2	50	**UNIQUE**
DOMICILE	VARCHAR2	30	—

Table 2: USER_ACCOUNT TABLE

Attribute	Data Type	Size	Constraints
USER_ID	NUMBER	-	PRIMARY KEY

STUDENT_ID	NUMBER	-	FOREIGN KEY REFERENCES STUDENT(STUDENT_ID), UNIQUE
USERNAME	VARCHAR	50	UNIQUE, NOT NULL
PASSWORD_HASH	VARCHAR	255	NOT NULL
ACCOUNT_STATUS	VARCHAR	20	CAN BE 'ACTIVE', 'INACTIVE', 'SUSPENDED'
CREATED_DATE	DATE	-	DEFAULT SYSDATE

TABLE 3: ACCOUNTS_OFFICE

Attribute	Data Type	Size	Constraints
OFFICE_ID	NUMBER	-	PRIMARY KEY
OFFICE_NAME	VARCHAR	100	NOT NULL
CAMPUS_ID	NUMBER	-	FOREIGN KEY REFERENCES CAMPUS(CAMPUS_ID)
CONTACT_PERSON	VARCHAR	100	-
CONTACT_NUMBER	VARCHAR	15	11-DIGIT NUMERIC PAKISTANI FORMAT

Table 4: CAMPUS

Attribute	Data Type	Size	Constraints
CAMPUS_ID	NUMBER	-	PRIMARY KEY
CAMPUS_NAME	VARCHAR	100	NOT NULL
LOCATION	VARCHAR	200	-

Table 5: DEPARTMENT

Attribute	Data Type	Size	Constraints
DEPT_ID	VARCHAR2	10	PRIMARY KEY
DEPT_NAME	VARCHAR2	50	UNIQUE, NOT NULL
CAMPUS_ID	NUMBER		FOREIGN KEY REFERENCES CAMPUS(CAMPUS_ID)

Table 6:PROGRAM

PROGRAM_ID	VARCHAR2	10	PRIMARY KEY
PROGRAM_NAME	VARCHAR2	50	NOT NULL
DEPT_ID	VARCHAR2	10	FOREIGN KEY REFERENCES DEPARTMENT(DEPT_ID)
TOTAL_SEATS	NUMBER	—	CHECK (TOTAL_SEATS > 0)
OPEN_MERIT_SEATS	NUMBER	—	CHECK (OPEN_MERIT_SEATS ≥ 0)
SELF_SUPPORT_SEATS	NUMBER	—	CHECK (SELF_SUPPORT_SEATS ≥ 0)

Table 7: SESSION

Attribute	Data Type	Size	Constraints
SESSION_ID	VARCHAR2	10	PRIMARY KEY
SESSION_YEAR	VARCHAR2	9	UNIQUE, NOT NULL (Format: '2023-2024')
START_DATE	DATE	-	NOT NULL
END_DATE	DATE	-	NOT NULL

Table 8: ADMISSION_CATEGORY

Attribute	Data Type	Size	Constraints
CATEGORY_ID	VARCHAR2	10	PRIMARY KEY
CATEGORY_NAME	VARCHAR2	30	UNIQUE, NOT NULL

Table 9: QUOTA

Attribute	Data Type	Size	Constraints
QUOTA_ID	VARCHAR2	10	PRIMARY KEY
QUOTA_TYPE	VARCHAR2	30	UNIQUE, NOT NULL
QUOTA_PERCENTAGE	NUMBER	5,2	CHECK (QUOTA_PERCENTAGE >= 0 AND QUOTA_PERCENTAGE <= 100)

TABLE 10:APPLICATION

Attribute	Data Type	Size	Constraints
APPLICATION_ID	VARCHAR2	15	PRIMARY KEY
STUDENT_ID	NUMBER		FOREIGN KEY REFERENCES STUDENT(STUDENT_ID)
SESSION_ID	VARCHAR2	10	FOREIGN KEY REFERENCES SESSIONS(SESSION_ID)
PROGRAM_ID	VARCHAR2	10	FOREIGN KEY REFERENCES PROGRAM(PROGRAM_ID)
CATEGORY_ID	VARCHAR2	10	FOREIGN KEY REFERENCES ADMISSION_CATEGORY(CATEGORY_ID)
QUOTA_ID	VARCHAR2	10	FOREIGN KEY REFERENCES QUOTA(QUOTA_ID)
APPLICATION_DATE	DATE	-	DEFAULT SYSDATE
STATUS	VARCHAR2	20	DEFAULT 'Submitted', CHECK (STATUS IN ('Submitted', 'Under Review', 'Approved', 'Rejected', 'Withdrawn'))

Table 11: ACADEMIC_MARKS

Attribute	Data Type	Size	Constraints
MARKS_ID	VARCHAR2	10	PRIMARY KEY
APPLICATION_ID	VARCHAR2	15	FOREIGN KEY REFERENCES APPLICATION(APPLICATION_ID)
MATRIC_TOTAL	NUMBER	6,2	CHECK (MATRIC_TOTAL > 0)
MATRIC_OBTAINED	NUMBER	6,2	CHECK (MATRIC_OBTAINED >= 0)
INTER_TOTAL	NUMBER	6,2	CHECK (INTER_TOTAL > 0)
INTER_OBTAINED	NUMBER	6,2	CHECK (INTER_OBTAINED >= 0)

Table 12 : TEST_SCORE

Attribute	Data Type	Size	Constraints
TEST_ID	VARCHAR2	10	PRIMARY KEY
APPLICATION_ID	VARCHAR2	15	FOREIGN KEY REFERENCES APPLICATION(APPLICATION_ID)
TEST_NAME	VARCHAR2	30	NOT NULL
TEST_DATE	DATE	-	NOT NULL
TOTAL_MARKS	NUMBER	5,2	CHECK (TOTAL_MARKS > 0)
OBTAINED_MARKS	NUMBER	5,2	CHECK (OBTAINED_MARKS >= 0)

Table 13: HAFIZ_CERTIFICATE

Attribute	Data Type	Size	Constraints
CERTIFICATE_ID	VARCHAR2	10	PRIMARY KEY
APPLICATION_ID	VARCHAR2	15	FOREIGN KEY REFERENCES APPLICATION(APPLICATION_ID)
CERTIFICATE_NUMBER	VARCHAR2	20	UNIQUE, NOT NULL
HAFIZ_VERIFIED	VARCHAR2	3	DEFAULT ‘No’, CHECK (HAFIZ_VERIFIED IN (‘Yes’, ‘No’))
VERIFIED_DATE	DATE	-	-

Table 14 : ADMIN

Attribute	Data Type	Size	Constraints
ADMIN_ID	VARCHAR2	10	PRIMARY KEY
USERNAME	VARCHAR2	20	UNIQUE, NOT NULL
ROLE	VARCHAR2	20	CHECK (ROLE IN (‘Super Admin’, ‘Admission Officer’, ‘Accounts Officer’, ‘Verification Officer’))
OFFICE_ID	NUMBER		FOREIGN KEY REFERENCES ACCOUNTS_OFFICE(OFFICE_ID)

Table 15: DOCUMENTS

Attribute	Data Type	Size	Constraints
DOC_ID	VARCHAR2	10	PRIMARY KEY
APPLICATION_ID	VARCHAR2	15	FOREIGN KEY REFERENCES APPLICATION(APPLICATION_ID)
DOC_TYPE	VARCHAR2	20	CHECK (DOC_TYPE IN ('Matric Certificate', 'Intermediate Certificate', 'CNIC', 'Domicile', 'Picture'))
FILE_PATH	VARCHAR2	200	NOT NULL
UPLOADED_DATE	DATE	-	DEFAULT SYSDATE
DOC_VERIFIED	VARCHAR2	3	DEFAULT 'No', CHECK (DOC_VERIFIED IN ('Yes', 'No'))
VERIFIED_BY	VARCHAR2	10	FOREIGN KEY REFERENCES ADMIN(ADMIN_ID)

Table 16: FEE_CHALLAN

Attribute	Data Type	Size	Constraints
CHALLAN_ID	VARCHAR2	15	PRIMARY KEY
APPLICATION_ID	VARCHAR2	15	FOREIGN KEY REFERENCES APPLICATION(APPLICATION_ID)
OFFICE_ID	NUMBER		FOREIGN KEY REFERENCES ACCOUNTS_OFFICE(OFFICE_ID)
FEE_TYPE	VARCHAR2	20	CHECK (FEE_TYPE IN ('Application Fee', 'Admission Fee', 'Tuition Fee', 'Hostel Fee'))
AMOUNT	NUMBER	10,2	CHECK (AMOUNT > 0)
DUE_DATE	DATE	-	NOT NULL
GENERATED_DATE	DATE	-	DEFAULT SYSDATE
STATUS	VARCHAR2	10	DEFAULT 'Pending', CHECK (STATUS IN ('Pending', 'Paid', 'Overdue', 'Cancelled'))

Table 17: PAYMENT

Attribute	Data Type	Size	Constraints
PAYMENT_ID	VARCHAR2	15	PRIMARY KEY
CHALLAN_ID	VARCHAR2	15	FOREIGN KEY REFERENCES FEE_CHALLAN(CHALLAN_ID)
PAYMENT_DATE	DATE	-	DEFAULT SYSDATE
AMOUNT_PAID	NUMBER	10,2	CHECK (AMOUNT_PAID > 0)

PAYMENT_METHOD	VARCHAR2	20	CHECK (PAYMENT_METHOD IN ('Bank Transfer', 'Credit Card', 'Debit Card', 'EasyPaisa', 'JazzCash'))
TRANSACTION_ID	VARCHAR2	30	UNIQUE, NOT NULL
PAYMENT_VERIFIED	VARCHAR2	3	DEFAULT 'No', CHECK (PAYMENT_VERIFIED IN ('Yes', 'No'))
VERIFIED_BY	VARCHAR2	10	FOREIGN KEY REFERENCES ADMIN(ADMIN_ID)

Table 18: MERIT

Attribute	Data Type	Size	Constraints
MERIT_ID	VARCHAR2	10	PRIMARY KEY
APPLICATION_ID	VARCHAR2	15	FOREIGN KEY REFERENCES APPLICATION(APPLICATION_ID)
MATRIC_WEIGHTAGE	NUMBER	5,2	CHECK (MATRIC_WEIGHTAGE >= 0 AND MATRIC_WEIGHTAGE <= 100)
INTER_WEIGHTAGE	NUMBER	5,2	CHECK (INTER_WEIGHTAGE >= 0 AND INTER_WEIGHTAGE <= 100)
TEST_WEIGHTAGE	NUMBER	5,2	CHECK (TEST_WEIGHTAGE >= 0 AND TEST_WEIGHTAGE <= 100)
HAFIZ_BONUS	NUMBER	5,2	DEFAULT 0, CHECK (HAFIZ_BONUS >= 0)
TOTAL_SCORE	NUMBER	6,2	CHECK (TOTAL_SCORE >= 0 AND TOTAL_SCORE <= 100)

Table 19: HOSTEL

Attribute	Data Type	Size	Constraints
HOSTEL_ID	VARCHAR2	10	PRIMARY KEY
APPLICATION_ID	VARCHAR2	15	FOREIGN KEY REFERENCES APPLICATION(APPLICATION_ID)
REQUEST_DATE	DATE	-	DEFAULT SYSDATE
HOSTEL_STATUS	VARCHAR2	15	DEFAULT 'Requested', CHECK (HOSTEL_STATUS IN ('Requested', 'Allocated', 'Waiting', 'Cancelled'))

Table 20: MERIT_LIST

Attribute	Data Type	Size	Constraints

LIST_ID	VARCHAR2	10	PRIMARY KEY
PROGRAM_ID	VARCHAR2	10	FOREIGN KEY REFERENCES PROGRAM(PROGRAM_ID)
SESSION_ID	VARCHAR2	10	FOREIGN KEY REFERENCES SESSION(SESSION_ID)
CATEGORY_ID	VARCHAR2	10	FOREIGN KEY REFERENCES ADMISSION_CATEGORY(CATEGORY_ID)
PUBLISHED_DATE	DATE	-	DEFAULT SYSDATE

Table 21: MERIT_RECORD

Attribute	Data Type	Size	Constraints
RECORD_ID	VARCHAR2	10	PRIMARY KEY
LIST_ID	VARCHAR2	10	FOREIGN KEY REFERENCES MERIT_LIST(LIST_ID)
MERIT_ID	VARCHAR2	10	FOREIGN KEY REFERENCES MERIT(MERIT_ID)
SELECTION_STATUS	VARCHAR2	15	DEFAULT 'Under Consideration', CHECK (SELECTION_STATUS IN ('Under Consideration', 'Selected', 'Waiting List', 'Not Selected'))
SELECTION_DATE	DATE	-	NOT NULL

Table 22: ADMISSION_OFFER

Attribute	Data Type	Size	Constraints
OFFER_ID	VARCHAR2	15	PRIMARY KEY
APPLICATION_ID	VARCHAR2	15	FOREIGN KEY REFERENCES APPLICATION(APPLICATION_ID)
OFFER_DATE	DATE	-	DEFAULT SYSDATE
ACCEPTANCE_DEADLINE	DATE	-	NOT NULL
OFFER_STATUS	VARCHAR2	15	DEFAULT 'Pending', CHECK (OFFER_STATUS IN ('Pending', 'Accepted', 'Rejected', 'Expired'))

5. TABLES CREATION USING SQL

1)STUDENT

```
CREATE TABLE STUDENT (
STUDENT_ID  NUMBER PRIMARY KEY,
CNIC     VARCHAR2(15) UNIQUE NOT NULL,
FULL_NAME   VARCHAR2(50) NOT NULL,
FATHER_NAME VARCHAR2(50),
DOB     DATE NOT NULL,
GENDER     VARCHAR2(10) CHECK (GENDER IN ('MALE','FEMALE')),
CONTACT    VARCHAR2(11)  CHECK (REGEXP_LIKE(CONTACT, '^03[0-9]{9}$')),
EMAIL      VARCHAR2(50) UNIQUE,
DOMICILE    VARCHAR2(30)
 };
```

Object Type **TABLE** Object **STUDENT**

Table	Column	Data Type	Length	Precision	Scale	Primary Key	Nullable	Default	Comment
STUDENT	STUDENT_ID	NUMBER	22	-	-	1	-	-	-
	CNIC	VARCHAR2	15	-	-	-	-	-	-
	FULL_NAME	VARCHAR2	50	-	-	-	-	-	-
	FATHER_NAME	VARCHAR2	50	-	-	-	✓	-	-
	DOB	DATE	7	-	-	-	-	-	-
	GENDER	VARCHAR2	10	-	-	-	✓	-	-
	CONTACT	VARCHAR2	11	-	-	-	✓	-	-
	EMAIL	VARCHAR2	50	-	-	-	✓	-	-
	DOMICILE	VARCHAR2	30	-	-	-	✓	-	-

1 - 9

2) USER_ACCOUNT

```
CREATE TABLE USER_ACCOUNT (
USER_ID INT PRIMARY KEY,
STUDENT_ID INT UNIQUE NOT NULL,
USERNAME VARCHAR(50) UNIQUE NOT NULL,
```

```
PASSWORD_HASH VARCHAR(255) NOT NULL,

ACCOUNT_STATUS VARCHAR(20) DEFAULT 'ACTIVE' CHECK (ACCOUNT_STATUS IN ('ACTIVE', 'INACTIVE', 'SUSPENDED')),

CREATED_DATE DATE DEFAULT CURRENT_DATE,

FOREIGN KEY (STUDENT_ID) REFERENCES STUDENT(STUDENT_ID)

);
```

Table	Column	Data Type	Length	Precision	Scale	Primary Key	Nullable	Default	Comment
USER_ACCOUNT	USER_ID	NUMBER	22	-	0	1	-	-	-
	STUDENT_ID	NUMBER	22	-	0	-	-	-	-
	USERNAME	VARCHAR2	50	-	-	-	-	-	-
	PASSWORD_HASH	VARCHAR2	255	-	-	-	-	-	-
	ACCOUNT_STATUS	VARCHAR2	20	-	-	-	✓	'ACTIVE'	-
	CREATED_DATE	DATE	7	-	-	-	✓	CURRENT_DATE	-

3) ACCOUNTS_OFFICE

```
CREATE TABLE ACCOUNTS_OFFICE (

OFFICE_ID NUMBER PRIMARY KEY,

OFFICE_NAME VARCHAR2(100) NOT NULL,

CAMPUS_ID NUMBER,

CONTACT_PERSON VARCHAR2(100),

CONTACT_NUMBER VARCHAR2(15),

FOREIGN KEY (CAMPUS_ID) REFERENCES CAMPUS(CAMPUS_ID),

CHECK (REGEXP_LIKE(CONTACT_NUMBER, '^03\d{9}$'))

);
```

Table	Column	Data Type	Length	Precision	Scale	Primary Key	Nullable	Default	Comment
ACCOUNTS_OFFICE	OFFICE_ID	NUMBER	22	-	-	1	-	-	-
	OFFICE_NAME	VARCHAR2	100	-	-	-	-	-	-
	CAMPUS_ID	NUMBER	22	-	0	-	✓	-	-
	CONTACT_PERSON	VARCHAR2	100	-	-	-	✓	-	-
	CONTACT_NUMBER	VARCHAR2	15	-	-	-	✓	-	-

4) CAMPUS

```
CREATE TABLE CAMPUS (

   CAMPUS_ID NUMBER PRIMARY KEY,
```

```
  CAMPUS_NAME VARCHAR(100) NOT NULL,
  LOCATION VARCHAR(200)
);
```

Table	Column	Data Type	Length	Precision	Scale	Primary Key	Nullable	Default	Comment
CAMPUS	CAMPUS_ID	NUMBER	22	-	0	1	-	-	-
	CAMPUS_NAME	VARCHAR2	100	-	-	-	-	-	-
	LOCATION	VARCHAR2	200	-	-	-	✓	-	-

5)DEPARTMENT

```
CREATE TABLE DEPARTMENT (
  DEPT_ID VARCHAR2(10) PRIMARY KEY,
  DEPT_NAME VARCHAR2(50) UNIQUE NOT NULL,
  CAMPUS_ID NUMBER  REFERENCES CAMPUS(CAMPUS_ID)
);
```

Table	Column	Data Type	Length	Precision	Scale	Primary Key	Nullable	Default	Comment
DEPARTMENT	DEPT_ID	VARCHAR2	10	-	-	1	-	-	-
	DEPT_NAME	VARCHAR2	50	-	-	-	-	-	-
	CAMPUS_ID	NUMBER	22	-	0	-	✓	-	-

6)PROGRAM

```
CREATE TABLE PROGRAM (
PROGRAM_ID    VARCHAR2(10) PRIMARY KEY,
PROGRAM_NAME     VARCHAR2(50) NOT NULL,
DEPT_ID    VARCHAR2(10),
TOTAL_SEATS       NUMBER CHECK (TOTAL_SEATS > 0),
OPEN_MERIT_SEATS   NUMBER CHECK (OPEN_MERIT_SEATS >= 0),
SELF_SUPPORT_SEATS NUMBER CHECK (SELF_SUPPORT_SEATS >= 0),
CONSTRAINT FK_PROGRAM_DEPT  FOREIGN KEY (DEPT_ID)  REFERENCES DEPARTMENT
(DEPT_ID),
CONSTRAINT CHK_SEATS
CHECK (TOTAL_SEATS = OPEN_MERIT_SEATS + SELF_SUPPORT_SEATS)
```

```
);
```

Table	Column	Data Type	Length	Precision	Scale	Primary Key	Nullable	Default	Comment
PROGRAM	PROGRAM_ID	VARCHAR2	10	-	-	1	-	-	-
	PROGRAM_NAME	VARCHAR2	50	-	-	-	-	-	-
	DEPT_ID	VARCHAR2	10	-	-	-	✓	-	-
	TOTAL_SEATS	NUMBER	22	-	-	-	✓	-	-
	OPEN_MERIT_SEATS	NUMBER	22	-	-	-	✓	-	-
	SELF_SUPPORT_SEATS	NUMBER	22	-	-	-	✓	-	-

7) SESSION

```
CREATE TABLE SESSIONS(
  SESSION_ID VARCHAR2(10) PRIMARY KEY,
  SESSION_YEAR VARCHAR2(9) UNIQUE NOT NULL,
  START_DATE DATE NOT NULL,
  END_DATE DATE NOT NULL
);
```

Table	Column	Data Type	Length	Precision	Scale	Primary Key	Nullable	Default	Comment
SESSIONS	SESSION_ID	VARCHAR2	10	-	-	1	-	-	-
	SESSION_YEAR	VARCHAR2	9	-	-	-	-	-	-
	START_DATE	DATE	7	-	-	-	-	-	-
	END_DATE	DATE	7	-	-	-	-	-	-

8) ADMISSION_CATEGORY

```
CREATE TABLE ADMISSION_CATEGORY (
  CATEGORY_ID VARCHAR2(10) PRIMARY KEY,
  CATEGORY_NAME VARCHAR2(30) UNIQUE NOT NULL
);
```

Table	Column	Data Type	Length	Precision	Scale	Primary Key	Nullable	Default	Comment
ADMISSION_CATEGORY	CATEGORY_ID	VARCHAR2	10	-	-	1	-	-	-
	CATEGORY_NAME	VARCHAR2	30	-	-	-	-	-	-

1 - 2

9) QUOTA

```
CREATE TABLE QUOTA (
  QUOTA_ID VARCHAR2(10) PRIMARY KEY,
```

```
  QUOTA_TYPE VARCHAR2(30) UNIQUE NOT NULL,
  QUOTA_PERCENTAGE NUMBER(5,2)   CHECK (QUOTA_PERCENTAGE >= 0 AND
QUOTA_PERCENTAGE <= 100)
);
```

Table	Column	Data Type	Length	Precision	Scale	Primary Key	Nullable	Default	Comment
QUOTA	QUOTA_ID	VARCHAR2	10	-	-	1	-	-	-
	QUOTA_TYPE	VARCHAR2	30	-	-	-	-	-	-
	QUOTA_PERCENTAGE	NUMBER	-	5	2	-	✓	-	-

1 - 3

10) APPLICATION

```
CREATE TABLE APPLICATION(
  APPLICATION_ID VARCHAR2(15) PRIMARY KEY,
  STUDENT_ID NUMBER REFERENCES STUDENT(STUDENT_ID),
  SESSION_ID VARCHAR2(10) REFERENCES SESSION(SESSION_ID),
  PROGRAM_ID VARCHAR2(10) REFERENCES PROGRAM(PROGRAM_ID),
  CATEGORY_ID VARCHAR2(10) REFERENCES ADMISSION_CATEGORY(CATEGORY_ID),
  QUOTA_ID VARCHAR2(10) REFERENCES QUOTA(QUOTA_ID),
  APPLICATION_DATE DATE DEFAULT SYSDATE,
  STATUS VARCHAR2(20) DEFAULT ‘SUBMITTED’   CHECK (STATUS IN (‘SUBMITTED’, ‘UNDER
REVIEW’, ‘APPROVED’, ‘REJECTED’, ‘WITHDRAWN’))
);
```

Table	Column	Data Type	Length	Precision	Scale	Primary Key	Nullable	Default	Comment
APPLICATION	APPLICATION_ID	VARCHAR2	15	-	-	1	-	-	-
	STUDENT_ID	NUMBER	22	-	-	-	✓	-	-
	SESSION_ID	VARCHAR2	10	-	-	-	✓	-	-
	PROGRAM_ID	VARCHAR2	10	-	-	-	✓	-	-
	CATEGORY_ID	VARCHAR2	10	-	-	-	✓	-	-
	QUOTA_ID	VARCHAR2	10	-	-	-	✓	-	-
	APPLICATION_DATE	DATE	7	-	-	-	✓	SYSDATE	-
	STATUS	VARCHAR2	20	-	-	-	✓	'Submitted'	-

1 - 8

11) ACADEMIC_MARKS

```
CREATE TABLE ACADEMIC_MARKS (
```

```
  MARKS_ID VARCHAR2(10) PRIMARY KEY,
  APPLICATION_ID VARCHAR2(15) REFERENCES APPLICATION(APPLICATION_ID),
  MATRIC_TOTAL NUMBER(6,2) CHECK (MATRIC_TOTAL > 0),
  MATRIC_OBTAINED NUMBER(6,2) CHECK (MATRIC_OBTAINED >= 0),
  INTER_TOTAL NUMBER(6,2) CHECK (INTER_TOTAL > 0),
  INTER_OBTAINED NUMBER(6,2) CHECK (INTER_OBTAINED >= 0)
);
```

Table	Column	Data Type	Length	Precision	Scale	Primary Key	Nullable	Default	Comment
ACADEMIC_MARKS	MARKS_ID	VARCHAR2	10	-	-	1	-	-	-
	APPLICATION_ID	VARCHAR2	15	-	-	-	✓	-	-
	MATRIC_TOTAL	NUMBER	-	6	2	-	✓	-	-
	MATRIC_OBTAINED	NUMBER	-	6	2	-	✓	-	-
	INTER_TOTAL	NUMBER	-	6	2	-	✓	-	-
	INTER_OBTAINED	NUMBER	-	6	2	-	✓	-	-

1 - 6

12) TEST_SCORE

```
CREATE TABLE TEST_SCORE (
  TEST_ID VARCHAR2(10) PRIMARY KEY,
  APPLICATION_ID VARCHAR2(15) REFERENCES APPLICATION(APPLICATION_ID),
  TEST_NAME VARCHAR2(30) NOT NULL,
  TEST_DATE DATE NOT NULL,
  TOTAL_MARKS NUMBER(5,2) CHECK (TOTAL_MARKS > 0),
  OBTAINED_MARKS NUMBER(5,2)  CHECK (OBTAINED_MARKS >= 0)
);
```

Table	Column	Data Type	Length	Precision	Scale	Primary Key	Nullable	Default	Comment
TEST_SCORE	TEST_ID	VARCHAR2	10	-	-	1	-	-	-
	APPLICATION_ID	VARCHAR2	15	-	-	-	✓	-	-
	TEST_NAME	VARCHAR2	30	-	-	-	-	-	-
	TEST_DATE	DATE	7	-	-	-	-	-	-
	TOTAL_MARKS	NUMBER	-	5	2	-	✓	-	-
	OBTAINED_MARKS	NUMBER	-	5	2	-	✓	-	-

13) HAFIZ_CERTIFICATE

```
CREATE TABLE HAFIZ_CERTIFICATE (
   CERTIFICATE_ID VARCHAR2(10) PRIMARY KEY,
   APPLICATION_ID VARCHAR2(15) REFERENCES APPLICATION(APPLICATION_ID),
   CERTIFICATE_NUMBER VARCHAR2(20) UNIQUE NOT NULL,
   HAFIZ_VERIFIED VARCHAR2(3) DEFAULT 'NO'  CHECK (HAFIZ_VERIFIED IN ('YES', 'NO')),
   VERIFIED_DATE DATE
);
```

Table	Column	Data Type	Length	Precision	Scale	Primary Key	Nullable	Default	Comment
HAFIZ_CERTIFICATE	CERTIFICATE_ID	VARCHAR2	10	-	-	1	-	-	-
	APPLICATION_ID	VARCHAR2	15	-	-	-	✓	-	-
	CERTIFICATE_NUMBER	VARCHAR2	20	-	-	-	-	-	-
	HAFIZ_VERIFIED	VARCHAR2	3	-	-	-	✓	'NO'	-
	VERIFIED_DATE	DATE	7	-	-	-	✓	-	-

1 - 5

14)ADMIN

```
CREATE TABLE ADMIN (
   ADMIN_ID VARCHAR2(10) PRIMARY KEY,
   USERNAME VARCHAR2(20) UNIQUE NOT NULL,
   ROLE VARCHAR2(20) CHECK (ROLE IN ('SUPER ADMIN', 'ADMISSION OFFICER', 'ACCOUNTS OFFICER', 'VERIFICATION OFFICER')),
   OFFICE_ID NUMBER  REFERENCES ACCOUNTS_OFFICE(OFFICE_ID)
);
```

Table	Column	Data Type	Length	Precision	Scale	Primary Key	Nullable	Default	Comment
ADMIN	ADMIN_ID	VARCHAR2	10	-	-	1	-	-	-
	USERNAME	VARCHAR2	20	-	-	-	-	-	-
	ROLE	VARCHAR2	20	-	-	-	✓	-	-
	OFFICE_ID	NUMBER	22	-	-	-	✓	-	-

1 - 4

15) DOCUMENTS

```
CREATE TABLE DOCUMENTS (
   DOC_ID VARCHAR2(10) PRIMARY KEY,
```

```
  APPLICATION_ID VARCHAR2(15) REFERENCES APPLICATION(APPLICATION_ID),
  DOC_TYPE VARCHAR2(20)
    CHECK (DOC_TYPE IN ('MATRIC CERTIFICATE', 'INTERMEDIATE CERTIFICATE', 'CNIC',
'DOMICILE', 'PICTURE')),
  FILE_PATH VARCHAR2(200) NOT NULL,
  UPLOADED_DATE DATE DEFAULT SYSDATE,
  DOC_VERIFIED VARCHAR2(3) DEFAULT 'NO'  CHECK (DOC_VERIFIED IN ('YES', 'NO')),
  VERIFIED_BY VARCHAR2(10) REFERENCES ADMIN(ADMIN_ID)
);
```

Table	Column	Data Type	Length	Precision	Scale	Primary Key	Nullable	Default	Comment
DOCUMENTS	DOC_ID	VARCHAR2	10	-	-	1	-	-	-
	APPLICATION_ID	VARCHAR2	15	-	-	-	✓	-	-
	DOC_TYPE	VARCHAR2	20	-	-	-	✓	-	-
	FILE_PATH	VARCHAR2	200	-	-	-	-	-	-
	UPLOADED_DATE	DATE	7	-	-	-	✓	SYSDATE	-
	DOC_VERIFIED	VARCHAR2	3	-	-	-	✓	'No'	-
	VERIFIED_BY	VARCHAR2	10	-	-	-	✓	-	-

16) FEE_CHALLAN

```
CREATE TABLE FEE_CHALLAN (
CHALLAN_ID VARCHAR2(15) PRIMARY KEY,
APPLICATION_ID VARCHAR2(15) REFERENCES APPLICATION(APPLICATION_ID),
OFFICE_ID NUMBER REFERENCES ACCOUNTS_OFFICE(OFFICE_ID),
FEE_TYPE VARCHAR2(20)  CHECK (FEE_TYPE IN ('APPLICATION FEE', 'ADMISSION FEE', 'TUITION
FEE', 'HOSTEL FEE')),
AMOUNT NUMBER(10,2) CHECK (AMOUNT > 0),
DUE_DATE DATE NOT NULL,
GENERATED_DATE DATE DEFAULT SYSDATE,
STATUS VARCHAR2(10) DEFAULT 'PENDING' CHECK (STATUS IN ('PENDING', 'PAID', 'OVERDUE',
'CANCELLED'))
);
```

Table	Column	Data Type	Length	Precision	Scale	Primary Key	Nullable	Default	Comment
FEE_CHALLAN	CHALLAN_ID	VARCHAR2	15	-	-	1	-	-	-
	APPLICATION_ID	VARCHAR2	15	-	-	-	✓	-	-
	OFFICE_ID	NUMBER	22	-	-	-	✓	-	-
	FEE_TYPE	VARCHAR2	20	-	-	-	✓	-	-
	AMOUNT	NUMBER	-	10	2	-	✓	-	-
	DUE_DATE	DATE	7	-	-	-	-	-	-
	GENERATED_DATE	DATE	7	-	-	-	✓	SYSDATE	-
	STATUS	VARCHAR2	10	-	-	-	✓	'PENDING'	-

1 - 8

17)PAYMENT

```
CREATE TABLE PAYMENT (
   PAYMENT_ID VARCHAR2(15) PRIMARY KEY,
   CHALLAN_ID VARCHAR2(15) REFERENCES FEE_CHALLAN(CHALLAN_ID),
   PAYMENT_DATE DATE DEFAULT SYSDATE,
   AMOUNT_PAID NUMBER(10,2) CHECK (AMOUNT_PAID > 0),
   PAYMENT_METHOD VARCHAR2(20)   CHECK (PAYMENT_METHOD IN (‘BANK TRANSFER’, ‘CREDIT CARD’, ‘DEBIT CARD’, ‘EASYPAISA’, ‘JAZZCASH’)),
   TRANSACTION_ID VARCHAR2(30) UNIQUE NOT NULL,
   PAYMENT_VERIFIED VARCHAR2(3) DEFAULT ‘NO’  CHECK (PAYMENT_VERIFIED IN (‘YES’, ‘NO’)),
   VERIFIED_BY VARCHAR2(10) REFERENCES ADMIN(ADMIN_ID)
);
```

Table	Column	Data Type	Length	Precision	Scale	Primary Key	Nullable	Default	Comment
PAYMENT	PAYMENT_ID	VARCHAR2	15	-	-	1	-	-	-
	CHALLAN_ID	VARCHAR2	15	-	-	-	✓	-	-
	PAYMENT_DATE	DATE	7	-	-	-	✓	SYSDATE	-
	AMOUNT_PAID	NUMBER	-	10	2	-	✓	-	-
	PAYMENT_METHOD	VARCHAR2	20	-	-	-	✓	-	-
	TRANSACTION_ID	VARCHAR2	30	-	-	-	-	-	-
	PAYMENT_VERIFIED	VARCHAR2	3	-	-	-	✓	'NO'	-
	VERIFIED_BY	VARCHAR2	10	-	-	-	✓	-	-

1 - 8

18)MERIT

```
CREATE TABLE MERIT (
```

```
  MERIT_ID VARCHAR2(10) PRIMARY KEY,

  APPLICATION_ID VARCHAR2(15) REFERENCES APPLICATION(APPLICATION_ID),

  MATRIC_WEIGHTAGE NUMBER(5,2)  CHECK (MATRIC_WEIGHTAGE >= 0 AND
MATRIC_WEIGHTAGE <= 100),

  INTER_WEIGHTAGE NUMBER(5,2)  CHECK (INTER_WEIGHTAGE >= 0 AND INTER_WEIGHTAGE <=
100),

  TEST_WEIGHTAGE NUMBER(5,2)  CHECK (TEST_WEIGHTAGE >= 0 AND TEST_WEIGHTAGE <=
100),

  HAFIZ_BONUS NUMBER(5,2) DEFAULT 0 CHECK (HAFIZ_BONUS >= 0),

  TOTAL_SCORE NUMBER(6,2)  CHECK (TOTAL_SCORE >= 0 AND TOTAL_SCORE <= 100),

);
```

Table	Column	Data Type	Length	Precision	Scale	Primary Key	Nullable	Default	Comment
MERIT	MERIT_ID	VARCHAR2	10	-	-	1	-	-	-
	APPLICATION_ID	VARCHAR2	15	-	-	-	✓	-	-
	MATRIC_WEIGHTAGE	NUMBER	-	5	2	-	✓	-	-
	INTER_WEIGHTAGE	NUMBER	-	5	2	-	✓	-	-
	TEST_WEIGHTAGE	NUMBER	-	5	2	-	✓	-	-
	HAFIZ_BONUS	NUMBER	-	5	2	-	✓	0	-
	TOTAL_SCORE	NUMBER	-	6	2	-	✓	-	-
									1 - 7

19)HOSTEL

```
CREATE TABLE HOSTEL (

  HOSTEL_ID VARCHAR2(10) PRIMARY KEY,

  APPLICATION_ID VARCHAR2(15) REFERENCES APPLICATION(APPLICATION_ID),

  REQUEST_DATE DATE DEFAULT SYSDATE,

  HOSTEL_STATUS VARCHAR2(15) DEFAULT ‘REQUESTED’  CHECK (HOSTEL_STATUS IN
(‘REQUESTED’, ‘ALLOCATED’, ‘WAITING’, ‘CANCELLED’))

);
```

Table	Column	Data Type	Length	Precision	Scale	Primary Key	Nullable	Default	Comment
HOSTEL	HOSTEL_ID	VARCHAR2	10	-	-	1	-	-	-
	APPLICATION_ID	VARCHAR2	15	-	-	-	✓	-	-
	REQUEST_DATE	DATE	7	-	-	-	✓	SYSDATE	-
	HOSTEL_STATUS	VARCHAR2	15	-	-	-	✓	'REQUESTED'	-

1 - 4

20) MERIT_LIST

```
CREATE TABLE MERIT_LIST (
   LIST_ID VARCHAR2(10) PRIMARY KEY,
   PROGRAM_ID VARCHAR2(10) REFERENCES PROGRAM(PROGRAM_ID),
   SESSION_ID VARCHAR2(10) REFERENCES SESSIONS(SESSION_ID),
   CATEGORY_ID VARCHAR2(10) REFERENCES ADMISSION_CATEGORY(CATEGORY_ID),
   PUBLISHED_DATE DATE DEFAULT SYSDATE
);
```

Table	Column	Data Type	Length	Precision	Scale	Primary Key	Nullable	Default	Comment
MERIT_LIST	LIST_ID	VARCHAR2	10	-	-	1	-	-	-
	PROGRAM_ID	VARCHAR2	10	-	-	-	✓	-	-
	SESSION_ID	VARCHAR2	10	-	-	-	✓	-	-
	CATEGORY_ID	VARCHAR2	10	-	-	-	✓	-	-
	PUBLISHED_DATE	DATE	7	-	-	-	✓	SYSDATE	-

1 - 5

21) MERIT_RECORD

```
CREATE TABLE MERIT_RECORD (
RECORD_ID VARCHAR2(10) PRIMARY KEY,
LIST_ID VARCHAR2(10) REFERENCES MERIT_LIST(LIST_ID),
MERIT_ID VARCHAR2(10) REFERENCES MERIT(MERIT_ID),
SELECTION_STATUS VARCHAR2(25)
DEFAULT ‘UNDER CONSIDERATION’
CHECK (SELECTION_STATUS IN
(‘UNDER CONSIDERATION’, ‘SELECTED’, ‘WAITING LIST’, ‘NOT SELECTED’)),
SELECTION_DATE DATE NOT NULL
```

```
);
```

Table	Column	Data Type	Length	Precision	Scale	Primary Key	Nullable	Default	Comment
MERIT_RECORD	RECORD_ID	VARCHAR2	10	-	-	1	-	-	-
	LIST_ID	VARCHAR2	10	-	-	-	✓	-	-
	MERIT_ID	VARCHAR2	10	-	-	-	✓	-	-
	SELECTION_STATUS	VARCHAR2	25	-	-	-	✓	'UNDER CONSIDERATION'	-
	SELECTION_DATE	DATE	7	-	-	-	-	-	-

1 - 5

22) ADMISSION_OFFER

```
CREATE TABLE ADMISSION_OFFER (

   OFFER_ID VARCHAR2(15) PRIMARY KEY,

   APPLICATION_ID VARCHAR2(15) REFERENCES APPLICATION(APPLICATION_ID),

   OFFER_DATE DATE DEFAULT SYSDATE,

   ACCEPTANCE_DEADLINE DATE NOT NULL,

 OFFER_STATUS VARCHAR2(15) DEFAULT 'PENDING'   CHECK (OFFER_STATUS IN ('PENDING', 'ACCEPTED', 'REJECTED', 'EXPIRED'))

);
```

Table	Column	Data Type	Length	Precision	Scale	Primary Key	Nullable	Default	Comment
ADMISSION_OFFER	OFFER_ID	VARCHAR2	15	-	-	1	-	-	-
	APPLICATION_ID	VARCHAR2	15	-	-	-	✓	-	-
	OFFER_DATE	DATE	7	-	-	-	✓	SYSDATE	-
	ACCEPTANCE_DEADLINE	DATE	7	-	-	-	-	-	-
	OFFER_STATUS	VARCHAR2	15	-	-	-	✓	'PENDING'	-

1 - 5

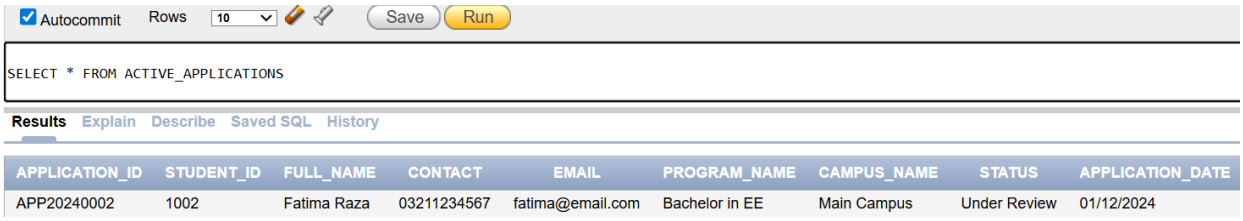
6. VIEWS CREATION USING SQL

1). ACTIVE_APPLICATIONS VIEW

Shows the Application Status along with Student Info and Program Details

```
CREATE OR REPLACE VIEW ACTIVE_APPLICATIONS AS

SELECT

   A.APPLICATION_ID,  S.STUDENT_ID,S.FULL_NAME,S.CONTACT, S.EMAIL,P.PROGRAM_NAME,

  C.CAMPUS_NAME,A.STATUS,  A.APPLICATION_DATE

FROM  APPLICATION A
```

```
JOIN
   STUDENT S ON A.STUDENT_ID = S.STUDENT_ID
JOIN
   PROGRAM P ON A.PROGRAM_ID = P.PROGRAM_ID
JOIN
   DEPARTMENT D ON P.DEPT_ID = D.DEPT_ID
JOIN
   CAMPUS C ON D.CAMPUS_ID = C.CAMPUS_ID
WHERE
   A.STATUS IN ('Submitted', 'Under Review');
```



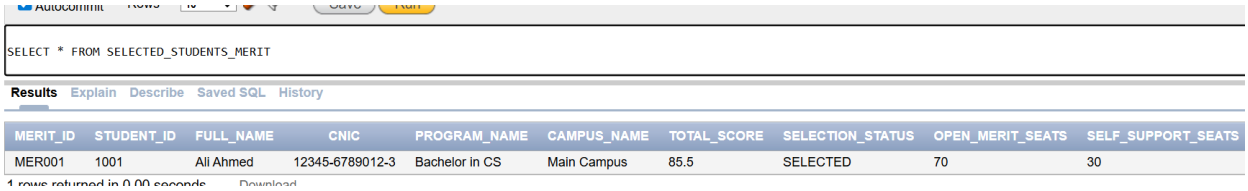
APPLICATION_ID	STUDENT_ID	FULL_NAME	CONTACT	EMAIL	PROGRAM_NAME	CAMPUS_NAME	STATUS	APPLICATION_DATE
APP20240002	1002	Fatima Raza	03211234567	fatima@email.com	Bachelor in EE	Main Campus	Under Review	01/12/2024

2).SELECTED STUDENTS MERIT VIEW

Shows the Merit Score and Ranking along with Selection Status and Program Seats.

```
CREATE OR REPLACE VIEW SELECTED_STUDENTS_MERIT AS
SELECT
   M.MERIT_ID,S.STUDENT_ID, S.FULL_NAME,  S.CNIC,P.PROGRAM_NAME, C.CAMPUS_NAME,
   M.TOTAL_SCORE, MR.SELECTION_STATUS,P.OPEN_MERIT_SEATS,  P.SELF_SUPPORT_SEATS
FROM MERIT M
JOIN
   APPLICATION A ON M.APPLICATION_ID = A.APPLICATION_ID
JOIN
   STUDENT S ON A.STUDENT_ID = S.STUDENT_ID
JOIN
   PROGRAM P ON A.PROGRAM_ID = P.PROGRAM_ID
```

```
JOIN
   DEPARTMENT D ON P.DEPT_ID = D.DEPT_ID
JOIN
   CAMPUS C ON D.CAMPUS_ID = C.CAMPUS_ID
JOIN
   MERIT_RECORD MR ON M.MERIT_ID = MR.MERIT_ID
WHERE
   MR.SELECTION_STATUS = 'SELECTED';
```



```
SELECT * FROM SELECTED_STUDENTS_MERIT
```

Results | Explain | Describe | Saved SQL | History

MERIT_ID	STUDENT_ID	FULL_NAME	CNIC	PROGRAM_NAME	CAMPUS_NAME	TOTAL_SCORE	SELECTION_STATUS	OPEN_MERIT_SEATS	SELF_SUPPORT_SEATS
MER001	1001	Ali Ahmed	12345-6789012-3	Bachelor in CS	Main Campus	85.5	SELECTED	70	30

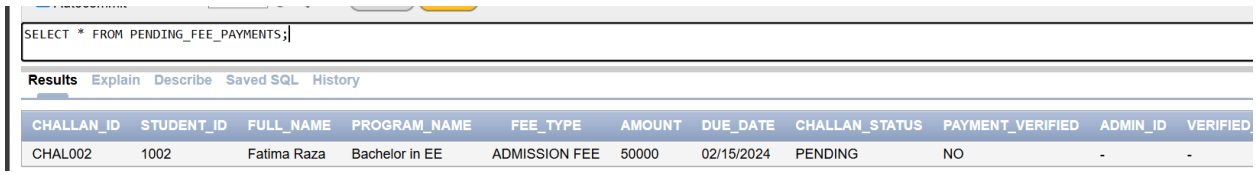
1 rows returned in 0.00 seconds Download

3)PENDING_FEE_PAYMENTS VIEW

Shows the Fee Payment Status along with Challan Amount and Payment Verification Info

```
CREATE OR REPLACE VIEW PENDING_FEE_PAYMENTS AS
SELECT
   FC.CHALLAN_ID,S.STUDENT_ID,S.FULL_NAME,P.PROGRAM_NAME, FC.FEE_TYPE,FC.AMOUNT,
   FC.DUE_DATE, FC.STATUS AS CHALLAN_STATUS, PY.PAYMENT_VERIFIED,A.ADMIN_ID,
   A.USERNAME AS VERIFIED_BY
FROM
   FEE_CHALLAN FC
JOIN
   APPLICATION APP ON FC.APPLICATION_ID = APP.APPLICATION_ID
JOIN
   STUDENT S ON APP.STUDENT_ID = S.STUDENT_ID
JOIN
   PROGRAM P ON APP.PROGRAM_ID = P.PROGRAM_ID
LEFT JOIN
```

```
   PAYMENT PY ON FC.CHALLAN_ID = PY.CHALLAN_ID
LEFT JOIN
   ADMIN A ON PY.VERIFIED_BY = A.ADMIN_ID
WHERE
   FC.STATUS IN ('PENDING', 'OVERDUE');
```



```
SELECT * FROM PENDING_FEE_PAYMENTS;
```

Results | Explain | Describe | Saved SQL | History

CHALLAN_ID	STUDENT_ID	FULL_NAME	PROGRAM_NAME	FEE_TYPE	AMOUNT	DUE_DATE	CHALLAN_STATUS	PAYMENT_VERIFIED	ADMIN_ID	VERIFIED
CHAL002	1002	Fatima Raza	Bachelor in EE	ADMISSION FEE	50000	02/15/2024	PENDING	NO	-	-

7. SELECT STATEMENTS FOR COMMON REPORTS

REPORT 1.STUDENT APPLICATIONS STATUS

```
SELECT A.APPLICATION_ID, S.STUDENT_ID, S.FULL_NAME AS STUDENT_NAME, S.CONTACT,
S.EMAIL, P.PROGRAM_NAME, C.CAMPUS_NAME,D.DEPT_NAME AS DEPARTMENT,
A.APPLICATION_DATE,A.STATUS AS APPLICATION_STATUS,
(SELECT COUNT(*) FROM APPLICATION A2
WHERE A2.STUDENT_ID = S.STUDENT_ID
AND UPPER(A2.STATUS) = 'SUBMITTED') AS TOTAL_SUBMITTED_APPS
FROM APPLICATION A
JOIN STUDENT S ON A.STUDENT_ID = S.STUDENT_ID
JOIN PROGRAM P ON A.PROGRAM_ID = P.PROGRAM_ID
JOIN DEPARTMENT D ON P.DEPT_ID = D.DEPT_ID
JOIN CAMPUS C ON D.CAMPUS_ID = C.CAMPUS_ID
WHERE UPPER(A.STATUS) IN ('SUBMITTED','UNDER REVIEW','UNDER_REVIEW')
ORDER BY A.APPLICATION_DATE DESC;
```

Results Explain Describe Saved SQL History

APPLICATION_ID	STUDENT_ID	STUDENT_NAME	CONTACT	EMAIL	PROGRAM_NAME	CAMPUS_NAME	DEPARTMENT	APPLICATION_DATE	APPLICATION_STATUS	TOTAL_SUB
APP20240002	1002	Fatima Raza	03211234567	fatima@email.com	Bachelor in EE	Main Campus	Electrical Engineering	01/12/2024	Under Review	0
APP20240001	1001	Ali Ahmed	03123456789	ali@email.com	Bachelor in CS	Main Campus	Computer Science	01/10/2024	Submitted	1

REPORT 2: FEE PAYMENT TRACKING

```
SELECT FC.CHALLAN_ID,S.FULL_NAME AS STUDENT,P.PROGRAM_NAME AS
PROGRAM,FC.FEE_TYPE,FC.AMOUNT AS DUE_AMOUNT, FC.DUE_DATE,FC.STATUS AS
CHALLAN_STATUS,PY.PAYMENT_DATE,PY.AMOUNT_PAID

FROM  FEE_CHALLAN FC

JOIN APPLICATION A ON FC.APPLICATION_ID = A.APPLICATION_ID

JOIN STUDENT S ON A.STUDENT_ID = S.STUDENT_ID

JOIN PROGRAM P ON A.PROGRAM_ID = P.PROGRAM_ID

LEFT JOIN PAYMENT PY ON FC.CHALLAN_ID = PY.CHALLAN_ID

ORDER BY FC.DUE_DATE, FC.STATUS;
```

Results Explain Describe Saved SQL History

CHALLAN_ID	STUDENT	PROGRAM	FEE_TYPE	DUE_AMOUNT	DUE_DATE	CHALLAN_STATUS	PAYMENT_DATE	AMOUNT_PAID
CHAL001	Ali Ahmed	Bachelor in CS	APPLICATION FEE	2000	02/01/2024	PAID	01/20/2024	2000
CHAL002	Fatima Raza	Bachelor in EE	ADMISSION FEE	50000	02/15/2024	PENDING	01/25/2024	50000

REPORT 3: HOSTEL REQUESTS

```
SELECT H.HOSTEL_ID,S.FULL_NAME AS STUDENT, S.GENDER,C.CAMPUS_NAME AS
CAMPUS,P.PROGRAM_NAME AS PROGRAM,H.HOSTEL_STATUS,H.REQUEST_DATE

FROM  HOSTEL H

JOIN APPLICATION A ON H.APPLICATION_ID = A.APPLICATION_ID

JOIN  STUDENT S ON A.STUDENT_ID = S.STUDENT_ID

JOIN PROGRAM P ON A.PROGRAM_ID = P.PROGRAM_ID

JOIN DEPARTMENT D ON P.DEPT_ID = D.DEPT_ID

JOIN CAMPUS C ON D.CAMPUS_ID = C.CAMPUS_ID
```

```
ORDER BY  C.CAMPUS_NAME, H.REQUEST_DATE DESC;
```

Results Explain Describe Saved SQL History

HOSTEL_ID	STUDENT	GENDER	CAMPUS	PROGRAM	HOSTEL_STATUS	REQUEST_DATE
HOST002	Fatima Raza	Female	Main Campus	Bachelor in EE	WAITING	02/22/2024
HOST001	Ali Ahmed	Male	Main Campus	Bachelor in CS	ALLOCATED	02/20/2024

REPORT 4: DOCUMENT VERIFICATION STATUS

```
SELECT DOC.DOC_ID,S.FULL_NAME AS
STUDENT,A.APPLICATION_ID,DOC.DOC_TYPE,DOC.UPLOADED_DATE,DOC.DOC_VERIFIED,AD.US
ERNAME AS VERIFIED_BY

FROM  DOCUMENTS DOC

JOIN APPLICATION A ON DOC.APPLICATION_ID = A.APPLICATION_ID

JOIN  STUDENT S ON A.STUDENT_ID = S.STUDENT_ID

LEFT JOIN  ADMIN AD ON DOC.VERIFIED_BY = AD.ADMIN_ID

ORDER BY  DOC.DOC_VERIFIED, DOC.UPLOADED_DATE;
```

Results Explain Describe Saved SQL History

DOC_ID	STUDENT	APPLICATION_ID	DOC_TYPE	UPLOADED_DATE	DOC_VERIFIED	VERIFIED_BY
DOC002	Fatima Raza	APP20240002	Matric Certificate	01/06/2024	No	-
DOC001	Ali Ahmed	APP20240001	CNIC	01/05/2024	Yes	admin1

REPOET 5.MERIT LIST AND SELECTION STATUS REPORT

```
SELECT M.MERIT_ID,S.FULL_NAME,P.PROGRAM_NAME,C.CAMPUS_NAME,
M.TOTAL_SCORE,A.STATUS

FROM MERIT M

JOIN APPLICATION A ON M.APPLICATION_ID = A.APPLICATION_ID

JOIN STUDENT S ON A.STUDENT_ID = S.STUDENT_ID

JOIN PROGRAM P ON A.PROGRAM_ID = P.PROGRAM_ID

JOIN DEPARTMENT D  ON P.DEPT_ID = D.DEPT_ID

JOIN CAMPUS C ON D.CAMPUS_ID = C.CAMPUS_ID

ORDER BY M.TOTAL_SCORE DESC;
```

Results Explain Describe Saved SQL History

MERIT_ID	FULL_NAME	PROGRAM_NAME	CAMPUS_NAME	TOTAL_SCORE	STATUS
MER001	Ali Ahmed	Bachelor in CS	Main Campus	85.5	Submitted
MER002	Fatima Raza	Bachelor in EE	Main Campus	78	Under Review

8. PROCEDURE,FUNCTIONS AND TRIGGERS

1. Procedure to Register a New Student

```
CREATE OR REPLACE PROCEDURE REGISTER_STUDENT(
  P_STUDENT_ID IN NUMBER,
  P_CNIC IN VARCHAR2,
  P_FULL_NAME IN VARCHAR2,
  P_FATHER_NAME IN VARCHAR2,
  P_DOB IN DATE,
  P_GENDER IN VARCHAR2,
  P_CONTACT IN VARCHAR2,
  P_EMAIL IN VARCHAR2,
  P_DOMICILE IN VARCHAR2
) IS
BEGIN
  INSERT INTO STUDENT (STUDENT_ID, CNIC, FULL_NAME, FATHER_NAME, DOB,
GENDER, CONTACT, EMAIL, DOMICILE)
  VALUES (P_STUDENT_ID, P_CNIC, P_FULL_NAME, P_FATHER_NAME, P_DOB,
P_GENDER, P_CONTACT, P_EMAIL, P_DOMICILE);
  DBMS_OUTPUT.PUT_LINE('STUDENT ' || P_FULL_NAME || ' REGISTERED
SUCCESSFULLY.');
END REGISTER_STUDENT;
```

```
BEGIN
    REGISTER_STUDENT(
        1003,'54321-9876543-1', 'HAMZA ALI', 'MUHAMMAD ALI',DATE '2001-06-20', 'Male',
        '03123456789', 'HAMZA.ALI@EMAIL.COM','KARACHI'
    );
END;
```

Results Explain Describe Saved SQL History

```
STUDENT HAMZA ALI REGISTERED SUCCESSFULLY.

Statement processed.
```

2. Procedure to Update Application Status

```
CREATE OR REPLACE PROCEDURE UPDATE_APPLICATION_STATUS(
  P_APPLICATION_ID IN VARCHAR2,P_STATUS IN VARCHAR2
) IS
BEGIN
  UPDATE APPLICATION
  SET STATUS = P_STATUS
  WHERE APPLICATION_ID = P_APPLICATION_ID;
  DBMS_OUTPUT.PUT_LINE('APPLICATION ' || P_APPLICATION_ID || ' STATUS UPDATED TO '
|| P_STATUS);
END UPDATE_APPLICATION_STATUS;
```

```
BEGIN
    UPDATE_APPLICATION_STATUS('APP2024001', 'APPROVED');
END;
```

Results Explain Describe Saved SQL History

```
APPLICATION APP2024001 STATUS UPDATED TO APPROVED

Statement processed.
```

3. Function to Calculate Final Merit Score for an Application

```
CREATE OR REPLACE FUNCTION CALCULATE_FINAL_MERIT(P_APPLICATION_ID VARCHAR2)
RETURN NUMBER
```

```
IS
 V_MATRIC NUMBER; V_INTER  NUMBER;  V_TEST   NUMBER; V_BONUS  NUMBER := 0; V_TOTAL NUMBER;
BEGIN
 SELECT (MATRIC_OBTAINED * 100.0 / MATRIC_TOTAL), (INTER_OBTAINED  * 100.0 / INTER_TOTAL)
   INTO V_MATRIC, V_INTER FROM ACADEMIC_MARKS
   WHERE APPLICATION_ID = P_APPLICATION_ID;
   SELECT (OBTAINED_MARKS * 100.0 / TOTAL_MARKS)
   INTO V_TEST FROM TEST_SCORE
   WHERE APPLICATION_ID = P_APPLICATION_ID;
   SELECT COUNT(*) INTO V_BONUS
   FROM HAFIZ_CERTIFICATE
   WHERE APPLICATION_ID = P_APPLICATION_ID
  AND HAFIZ_VERIFIED = 'YES';
   IF V_BONUS > 0 THEN V_BONUS := 20; END IF;
   V_TOTAL := (V_MATRIC*0.10) + (V_INTER*0.40) + (V_TEST*0.50) + V_BONUS;
   IF V_TOTAL > 100 THEN V_TOTAL := 100; END IF;
   RETURN ROUND(V_TOTAL,2);
EXCEPTION
   WHEN NO_DATA_FOUND THEN
      RETURN 0;
END;
```

```
SELECT CALCULATE_FINAL_MERIT('APP20240001') AS FINAL_MERIT FROM DUAL;
```

Results Explain Describe Saved SQL History

FINAL_MERIT
87.02

4. Function to check seat availability

```
CREATE OR REPLACE FUNCTION CHECK_SEAT_AVAILABILITY(
P_PROGRAM_ID VARCHAR2,P_CATEGORY_ID VARCHAR2)
RETURN VARCHAR2
IS
V_TOTAL_SEATS     NUMBER;
V_ALLOCATED_SEATS  NUMBER;
V_AVAILABLE_SEATS  NUMBER;
BEGIN
SELECT
CASE
WHEN P_CATEGORY_ID = 'CAT001' THEN OPEN_MERIT_SEATS
WHEN P_CATEGORY_ID = 'CAT002' THEN SELF_SUPPORT_SEATS
ELSE TOTAL_SEATS
END
INTO V_TOTAL_SEATS
FROM PROGRAM  WHERE PROGRAM_ID = P_PROGRAM_ID;
SELECT COUNT(*)  INTO V_ALLOCATED_SEATS
FROM APPLICATION A
JOIN ADMISSION_OFFER AO ON A.APPLICATION_ID = AO.APPLICATION_ID
WHERE A.PROGRAM_ID = P_PROGRAM_ID AND A.CATEGORY_ID = P_CATEGORY_ID
AND AO.OFFER_STATUS = 'ACCEPTED';
V_AVAILABLE_SEATS := V_TOTAL_SEATS - V_ALLOCATED_SEATS;
IF V_AVAILABLE_SEATS > 0 THEN
RETURN 'AVAILABLE: ' || V_AVAILABLE_SEATS || ' SEATS';
ELSE
RETURN 'NO SEATS AVAILABLE';
```

```
END IF;
EXCEPTION
WHEN NO_DATA_FOUND THEN
RETURN 'PROGRAM NOT FOUND';
WHEN OTHERS THEN
RETURN 'ERROR CHECKING AVAILABILITY';
END CHECK_SEAT_AVAILABILITY;
```

```
SELECT CHECK_SEAT_AVAILABILITY('PROG001', 'CAT001') FROM DUAL;
```

Results | Explain | Describe | Saved SQL | History

CHECK_SEAT_AVAILABILITY('PROG001','CAT001')
PROGRAM NOT FOUND

5. Trigger to Validate Fee Payment Before Status Update

```
CREATE OR REPLACE TRIGGER FEE_CHECK
BEFORE UPDATE ON APPLICATION
FOR EACH ROW
DECLARE
 V_DUE  NUMBER;
V_PAID NUMBER;
BEGIN
IF :NEW.STATUS = 'APPROVED' AND :OLD.STATUS <> 'APPROVED' THEN
SELECT SUM(FC.AMOUNT), SUM(NVL(P.AMOUNT_PAID,0))
INTO V_DUE, V_PAID FROM FEE_CHALLAN FC
LEFT JOIN PAYMENT P ON FC.CHALLAN_ID = P.CHALLAN_ID
WHERE FC.APPLICATION_ID = :NEW.APPLICATION_ID;
IF V_DUE IS NULL OR V_PAID < V_DUE THEN
RAISE_APPLICATION_ERROR(-20001,'FEES MUST BE FULLY PAID BEFORE APPROVAL');
```

```
 END IF;
   END IF;
END FEE_CHECK;
```

6.Trigger to Validate Student Age Before Application

```
CREATE OR REPLACE TRIGGER TRG_VALIDATE_STUDENT_AGE
BEFORE INSERT ON APPLICATION
FOR EACH ROW
DECLARE
   V_STUDENT_DOB DATE;
   V_AGE_YEARS NUMBER;
BEGIN
   SELECT DOB INTO V_STUDENT_DOB
   FROM STUDENT  WHERE STUDENT_ID = :NEW.STUDENT_ID;
   V_AGE_YEARS := TRUNC(MONTHS_BETWEEN(SYSDATE, V_STUDENT_DOB) / 12);
   IF V_AGE_YEARS < 16 THEN
      RAISE_APPLICATION_ERROR(-20001, 'STUDENT MUST BE AT LEAST 16 YEARS OLD TO
APPLY. CURRENT AGE: ' || V_AGE_YEARS);
   END IF;
   IF V_AGE_YEARS > 30 THEN
      DBMS_OUTPUT.PUT_LINE('WARNING: STUDENT IS OVER 30 YEARS OLD (AGE: ' ||
V_AGE_YEARS || ')');
   END IF;
END TRG_VALIDATE_STUDENT_AGE;
```